

Evaluating and Enhancing Code Language Models with Retrieval-Augmented Generation

Full Capstone Report — Project, Delivery & Individual Contributions

AIML PGCP Capstone

Group Number: 39 • Batch: 26

TalentSprint / Accenture

Team Members:

Mahin Nandipa • Abhinaya Thavishi • Ashu Bagul

Mentors: Pawan Baswani, Nayan Anand

Supervisor: Prof. Anil Neelkanti

July 2026

Table of Contents

Abstract.....	3
1. Problem Statement.....	3
1.1 Objectives.....	3
2. Literature Review.....	4
2.1 Spider vs. Spider 2.0 — A Resolved Discrepancy.....	4
3. Methodology.....	4
3.1 System Architecture.....	4
3.2 Evaluation Methodology.....	5
3.3 Delivery Methodology.....	5
4. Outcomes in Stages.....	5
4.1 – 4.4 Stage 1–4 Outcomes (Checkpoints 1–4).....	5
4.5 Post-Deployment Engineering Outcomes.....	6
5. Final Outcomes.....	6
5.1 Baseline Comparison Table.....	6
5.2 Discussion.....	7
5.3 Deployment Verification.....	7
5.4 Publishing Outcomes.....	7
6. Challenges.....	7
6.1 – 6.5 Infrastructure, Access & Scope Issues.....	7
7. Applicability in the Real World.....	8
7.1 – 7.6 Real-World Use Cases.....	8
8. Team & Individual Contributions.....	9
8.1 Team Contributions (Checkpoints 1–4).....	9
8.2 Individual Contribution — Mahin Nandipa.....	9
9. Limitations and Gap-Closure Note.....	10
9.1 Checkpoint 4 Gap — Closed at the Code Level.....	11
9.2 Spider vs. Spider 2.0 — Resolved.....	11
9.3 Remaining Open Item.....	11
10. Conclusion and Future Work.....	11
11. References.....	12

Abstract

This report is a complete account of the CodeGen RAG Capstone project: the research system built across four checkpoints, the engineering work carried out to synchronize, repair, test, and publish that system, and the individual contributions made by each part of the effort — including a dedicated account of this author's (Mahin Nandipa's) specific contributions. The underlying project evaluates and enhances Salesforce/codegen-350M-multi, a small open-source code language model, across five software-engineering tasks — program synthesis, documentation generation, commit message generation, code translation, and natural-language-to-SQL generation — comparing it against a Rust-fine-tuned variant, an upper-bound LLM (Claude Sonnet 4 with GPT-5 fallback), and a Retrieval-Augmented Generation pipeline layered on that upper-bound LLM. Beyond the original research scope, this report also documents the deployment-hardening phase: repository synchronization, two functional bug fixes, three infrastructure issues diagnosed and resolved, test growth from 108 to 147 passing tests, publication to two GitHub repositories, and documentation consolidation.

1. Problem Statement

Small, openly licensed code language models (hundreds of millions of parameters, runnable without specialized infrastructure) lag behind proprietary large language models on most code-generation benchmarks, but are far cheaper to fine-tune, deploy, and run at scale. This gap raises two open questions this project sets out to answer. First, how much of that gap can be closed for a specific, underrepresented target — in this case Rust, a language absent from codegen-350M-multi's original pretraining corpus — through targeted fine-tuning alone, without changing the base model's parameter count. Second, how much additional quality can be recovered, for both the small model and an upper-bound LLM, by augmenting generation with retrieval over a relevant code corpus, and whether combining dense semantic retrieval with structural (AST-based) retrieval outperforms either technique alone.

1.1 Objectives

- Evaluate codegen-350M-multi as a baseline across program synthesis, documentation generation, commit message generation, code translation, and text-to-SQL generation, using automatic metrics validated against human-judgment-correlated methods (CodeBLEU, CodeBERTScore, execution accuracy, pass@k).
- Extend the base model to Rust via parameter-efficient (LoRA) and full fine-tuning, and route deployment traffic to the best available fine-tuned checkpoint automatically.
- Build a Retrieval-Augmented Generation pipeline combining dense embedding search (FAISS) and AST-structural search, fused via reciprocal rank fusion, and measure its effect across model tiers.
- Deploy the complete system — base model, fine-tuned model, RAG pipeline, and upper-bound LLM comparison — behind a documented, containerized, and tested API and demo UI, then verify and publish that system for team and mentor review.

2. Literature Review

This project builds directly on twelve prior works, summarized in full (with corrected citation details) in the companion Research Paper Summaries document. The base model architecture and multi-turn synthesis

framing follow CodeGen (Nijkamp et al., 2022), a family of open autoregressive code models trained across multiple programming languages. Functional-correctness evaluation follows the HumanEval / pass@k methodology introduced in Codex (Chen et al., 2021), the OpenAI paper that also powers GitHub Copilot. CodeBERT (Feng et al., 2020) underlies the general encoder-based similarity approach informing this project's dense retrieval design and evaluation.

The text-to-SQL evaluation setup draws on two complementary benchmarks: Spider (Yu et al., 2018), a cross-domain benchmark requiring generalization to unseen database schemas, and BIRD (Li et al., 2023, corrected from the project README's stated 2024), which tests robustness against large, messy, real-world production databases. Documentation-generation evaluation is informed by CoDocBench (Pai et al., 2024, corrected from the README's stated "Pal"), a dataset of paired code-and-documentation snapshots from real commit history.

The RAG architecture itself follows Lewis et al. (2020), which established retrieval-augmented generation as a general pattern combining a parametric generator with a non-parametric retrieval memory, with ReACC (Lu et al., 2021) as the closest prior work combining sparse/dense retrieval specifically with code generation. Automatic evaluation throughout the project relies on CodeBLEU (Ren et al., 2020) and CodeBERTScore (Zhou et al., 2023) as complementary structural and semantic similarity metrics — the former sensitive to AST and dataflow correctness, the latter to soft semantic equivalence that survives superficial rewording.

One citation in the project README, "AST retrieval (Jiang et al., 2023)," could not be verified against a specific matching publication during this report's preparation and should be confirmed against the original project proposal before external submission — this is flagged transparently rather than fabricating a source.

2.1 Spider vs. Spider 2.0 — A Resolved Discrepancy

The official project brief's reference list links to spider2-sql.github.io — Spider 2.0, a substantially newer and harder benchmark built against live enterprise data warehouses (BigQuery, Snowflake, DuckDB) rather than a fixed set of downloadable SQLite databases. This project's actual implementation was checked directly against its source code for this report: `src/codegen_rag/data/downloaders.py` downloads the `xlangai/spider` dataset from Hugging Face — the classic 2018 Spider benchmark (Yu et al.), with 200 cross-domain databases, matching the citation already used in the project README and throughout this report's results. This is very likely a reference-list error in the brief rather than a defect in the project, since classic Spider is what "Spider" conventionally refers to in this kind of coursework and Spider 2.0's live-warehouse setup would require an entirely different, much larger-scope data and evaluation pipeline. The recommendation is a one-line confirmation with the mentors that classic Spider was the intended benchmark, rather than a rebuild against Spider 2.0.

3. Methodology

3.1 System Architecture

The system is organized into six layers that share a single typed Settings object end to end, so that all four project checkpoints operate on one coherent pipeline rather than four disconnected demos. The data layer downloads and normalizes five sources: CoDocBench (code/documentation/commit-message triples), Spider and BIRD (text-to-SQL question/schema/query triples with SQLite database archives),

CodeParrot (the RAG corpus and general fine-tuning source), and a filtered Rust subset of CodeParrot for the language-extension task.

The model layer wraps Salesforce/codegen-350M-multi behind a single CodeGenModel class that handles text generation, embedding extraction, and LoRA adapter loading. Task modules (program synthesis, documentation generation, commit message generation, code translation, and SQL generation) are thin subclasses of a common BaseTask. The RAG pipeline embeds the corpus into a FAISS index, separately indexes AST node-type sequences for structural retrieval, and fuses both signals via Reciprocal Rank Fusion. Deployment exposes the identical task modules and RAG pipeline through FastAPI, with a Streamlit UI as a thin HTTP client on top; both are containerized and wired together via docker-compose.

3.2 Evaluation Methodology

Every generation task is scored with a combination of exact match, CodeBLEU (structural and semantic similarity via weighted n-gram, AST, and dataflow match), and CodeBERTScore-style BERTScore F1. SQL generation is additionally scored with execution accuracy, and program synthesis is scored with pass@k where sampling budget allows.

3.3 Delivery Methodology

Beyond the research methodology above, this project also followed a deliberate delivery methodology once the four checkpoints were functionally complete: verify that exploratory sandbox fixes actually reached the team's live Google Drive repository; root-cause and fix any gaps found rather than assume they were cosmetic; grow the automated test suite to lock in every fix; publish the verified repository to both a personal and a shared mentor GitHub repository; and consolidate documentation so the finished work is easy for graders and teammates to review. Sections 4 through 8 report the outcomes of that methodology in detail.

4. Outcomes in Stages

The project was executed across four checkpoints, each additive on the last, sharing the same Settings object, model wrapper, and results directory. This section reports what was concretely produced at each stage, followed by the additional post-deployment engineering outcomes delivered afterward.

4.1 Stage 1 (Checkpoint 1) — Foundation, Baseline Evaluation, Rust LoRA

Established the environment bootstrap, downloaded CoDocBench and a Rust corpus subset, and evaluated the untouched base model on four tasks.

- Outcome: the project's first fine-tuned checkpoint — a LoRA adapter trained on a 500-sample Rust subset — plus baseline metrics for all four tasks.

4.2 Stage 2 (Checkpoint 2) — SQL Generation and Full Fine-Tuning

Downloaded the full Spider and BIRD datasets with their SQLite database archives, implemented schema-aware SQL generation, and replaced the Stage 1 LoRA subset run with a full-corpus Rust fine-tune.

- Outcome: a full-corpus Rust fine-tuned checkpoint and real execution-accuracy scores for text-to-SQL generation on both Spider and BIRD.

4.3 Stage 3 (Checkpoint 3) — Retrieval-Augmented Generation

Embedded a corpus of over 2,000 samples, built a FAISS dense index and a separate AST structural index, and swept retrieval strategy and Top-K to find the best-performing configuration.

- Outcome: the best retrieval configuration identified from the sweep, and the project's first real, non-placeholder four-tier comparison table.

4.4 Stage 4 (Checkpoint 4) — Deployment

Exposed every task module and the RAG pipeline through a FastAPI backend and a four-tab Streamlit demo UI, containerized and orchestrated via docker-compose.

- Outcome: a live, containerized, tested deployment — the same demo used for the final presentation.

4.5 Post-Deployment Engineering Outcomes

Two further engineering gaps were identified and closed during deployment hardening. First, checkpoint-aware model routing: every deployment endpoint had been serving the base model regardless of whether a trained Rust checkpoint existed. This was corrected with a resolution layer that locates the best available checkpoint per language. Second, genuine AST/dataflow-aware CodeBLEU: without a Python grammar available, CodeBLEU's structural components were silently falling back to a plain text-similarity approximation. Adding tree-sitter-python at a version verified compatible with the pinned toolchain restored real structural scoring.

- Outcome: deployment automatically serves the best available fine-tuned Rust checkpoint (confirmed live via the Streamlit demo), and CodeBLEU scores reflect genuine structural similarity. Test coverage grew from 108 to 147 passing tests to lock in both fixes.

5. Final Outcomes

5.1 Baseline Comparison Table

The table below reports the small-LM baseline tier's results from the project's own comparison_table.csv, generated by the Checkpoint 3 evaluation run — the first real, non-placeholder numbers produced once the notebooks were run end to end in Colab with a GPU.

Task	Model Tier	N	Exact Match	CodeBLEU	BERTScore F1	Exec. Accuracy
sql_generation_spider	small_lm_baseline	200	—	—	—	0.07
sql_generation_birdbench	small_lm_baseline	200	—	—	—	0.00
program_synthesis	small_lm_baseline	100	0.00	0.1046	0.8762	—
commit_message_generator	small_lm_baseline	100	0.00	0.0002	0.8167	—

Task	Model Tier	N	Exact Match	CodeBLEU	BERTScore F1	Exec. Accuracy
ation	ne					
code_translation	small_lm_baseline	29	0.00	0.0424	0.8194	—
documentation_generation	small_lm_baseline	100	0.00	0.0217	—	—

5.2 Discussion

The near-zero exact-match scores across every task are expected for open-ended code and text generation. Program synthesis's CodeBLEU (0.1046) and BERTScore F1 (0.8762) are the small baseline's strongest showing. Commit message generation's very low CodeBLEU (0.0002) alongside a comparatively high BERTScore F1 (0.8167) illustrates why both metrics are reported together. Both SQL benchmarks show low execution accuracy at the small-LM baseline tier, matching the literature's expectation that BIRD's real-world database complexity is substantially harder than Spider's cleaner cross-domain schemas.

5.3 Deployment Verification

The final deployed system was verified live: the FastAPI backend's health endpoint and the Streamlit demo UI both confirm the Rust-language endpoint is served by the fine-tuned checkpoint rather than the base model, and the full 147-test pytest suite passes in under five seconds with no GPU, network access, or downloaded model weights required.

5.4 Publishing Outcomes

The verified, 147-test-passing repository was published to two GitHub locations: a personal repository (mahin-aeroai/CodeGen-) and the mentor-shared collaboration repository (nayanjha16/CodeGen-Implementations-May_26) on its Group-39 branch, with the full project documentation consolidated into that branch's default-visible README.md to match the depth of a peer team's example.

6. Challenges

Deploying and iterating on this system live in Google Colab surfaced three infrastructure issues beyond ordinary application bugs, each worth documenting because the symptoms were initially indistinguishable from a code-level regression.

6.1 Google Drive FUSE Write-Sync Race

A file write to a Drive-mounted path can succeed and self-verify within the same Colab session without having actually flushed to persistent storage, and a Runtime restart shortly afterward can silently revert it. This was root-caused through repeated MD5 re-verification across restarts, and fixed by rewriting every deployment patch cell to retry its write with a short delay and a full read-back-and-hash comparison rather than a single write treated as final.

6.2 Stale Compiled Bytecode on a Drive-Mounted Package

Google Drive's FUSE mount can report modification times unreliably, causing Python to keep using an old compiled .pyc file even after the corresponding source file was confirmed correct — surfacing as an ImportError for a function demonstrably present in the source. Fixed by recursively deleting every __pycache__ directory before the next Runtime restart.

6.3 Checkpoint Directory Nesting

The checkpoint-routing feature initially checked for a checkpoint's marker files one directory level too shallow, because the checkpoint manager nests every save under a checkpoint-NNNNNN subdirectory. Fixed with an explicit checkpoint-resolution helper and eight dedicated unit tests.

6.4 Repository and Access Issues

Two further, non-technical challenges arose during publishing. First, a repo-path mismatch: the project's own settings.root_dir resolved to a different Drive folder than the actual git-tracked source repository, discovered by comparing pytest's printed rootdir against the assumed path. Second, a GitHub push initially failed with a 403 permission error, diagnosed as an insufficient Personal Access Token scope rather than a missing repository, and resolved by regenerating the token with the repo scope explicitly enabled.

6.5 Scope and Resource Constraints

The development environment used to write and test the code had no GPU, no live API access, and no practical way to download the multi-gigabyte datasets — every module was instead unit- and integration-tested in isolation with fakes standing in for real model and network calls, with the first true end-to-end numbers produced only once the notebooks were run in Colab with a GPU and valid API keys.

7. Applicability in the Real World

The techniques demonstrated in this project map directly onto problems organizations face today when adopting AI-assisted software development, beyond their role as an academic exercise.

7.1 Cost-Efficient Developer Tooling

Small, self-hosted code models are dramatically cheaper to run at scale than proprietary frontier LLMs. This project's four-tier comparison methodology is directly reusable by any organization deciding whether a smaller, self-hosted model is good enough for a specific internal use case.

7.2 Extending Coverage to Underrepresented Languages

The Rust fine-tuning workflow generalizes to other underrepresented or organization-specific languages: legacy enterprise languages (COBOL, Fortran), internal domain-specific languages, or newer languages not yet well represented in public pretraining corpora.

7.3 Natural-Language Database Access

The text-to-SQL capability directly supports letting non-technical stakeholders query databases in plain English, evaluated the way that matters for safety: execution accuracy, not superficial query similarity.

7.4 Retrieval-Augmented Generation Grounded in Proprietary Code

The hybrid dense-plus-AST retrieval pipeline is directly transferable to grounding a code assistant in an organization's own codebase, reducing hallucinated APIs without retraining the underlying model.

7.5 Developer Productivity Automation

Documentation-generation and commit-message-generation directly automate two of the most commonly deprioritized developer chores.

7.6 A Reusable Deployment Blueprint

The FastAPI-plus-Streamlit-plus-Docker deployment pattern, with checkpoint-aware model routing, is a directly reusable blueprint for standing up an internal developer-tooling service.

8. Team & Individual Contributions

This section records who did what, both at the team level across the four research checkpoints and, in detail, this author's individual contribution during the delivery and publishing phase of the project.

8.1 Team Contributions (Checkpoints 1–4)

The four-checkpoint research pipeline — environment bootstrap, dataset integration, task modules, model fine-tuning, the retrieval-augmented generation pipeline, and the FastAPI/Streamlit deployment — was built collaboratively by the full team (Mahin Nandipa, Abhinaya Thavishi, and Ashu Bagul) across the project's four checkpoint milestones, under the guidance of mentors Pawan Baswani and Nayan Anand and supervisor Prof. Anil Neelkanti. Commit history on the shared mentor repository's Group-39 branch reflects ongoing contributions from multiple team members.

8.2 Individual Contribution — Mahin Nandipa

The following contributions in the delivery and publishing phase of the project were driven and executed directly by this author, working from the team's existing four-checkpoint codebase to verify, repair, test, and publish it, and to prepare the final academic deliverables.

Area	Contribution
Repository audit	Identified that exploratory sandbox fixes had not reached the team's live Google Drive repository, and that the repository's settings.root_dir pointed to a different Drive folder than its actual git-tracked source, requiring every subsequent fix to target the confirmed-correct path.
Test suite growth	Diagnosed a 22-test gap by diffing pytest collection output against the team's own verbose run, then authored and delivered a combined, MD5-verified sync patch for 10 files, growing the suite from 108 to 147 passing tests.
Checkpoint-aware model routing	Root-caused why deployment endpoints were silently serving the base model instead of the trained Rust checkpoint (a directory-nesting bug), and authored the fix plus its eight dedicated unit tests.

Area	Contribution
Genuine AST-aware CodeBLEU	Identified that CodeBLEU's structural scoring was silently falling back to a plain text-similarity proxy, and resolved it by pinning a compatible tree-sitter-python version.
Infrastructure diagnostics	Root-caused two further non-obvious Colab/Drive issues — a file write-sync race and stale compiled bytecode — and authored the retry-with-verification pattern used across every subsequent patch cell.
Missing .gitignore	Discovered that the repository's .gitignore was completely absent (despite being assumed present), authored a replacement, and cleaned __pycache__ contamination out of git history.
LICENSE	Authored and added an MIT LICENSE file, credited to the group collectively.
GitHub publishing	Diagnosed and resolved a 403 push failure (insufficient PAT scope) and published the verified repository to a personal GitHub repository.
Mentor repository publishing	Published the verified repository to the mentor-shared collaboration repository's Group-39 branch without disrupting existing team commit history, using group-level (non-personal) commit attribution per the team's request.
README consolidation	Merged the full project documentation directly into the Group-39 branch's default-visible README.md, matching the depth of a peer team's reference example, while preserving the branch's existing team README content.
Final deliverable authoring	Authored and formatted this report, the accompanying Presentation Slides deck (20 slides with full speaker notes), and the Research Paper Summaries document, restructuring all three to match the official capstone submission rubric and correcting two citation errors found in the process.

9. Limitations and Gap-Closure Note

This repository was built to be fully correct and runnable, but portions of it were not executed against a GPU, the live Claude Sonnet 4 / GPT-5 APIs, or the multi-gigabyte datasets inside the development sandbox used to write and test the code. Every component was instead unit- or integration-tested in isolation with fakes standing in for real model and network calls (153 of 153 tests passing as of this report, up from 147 — see Section 9.1). The baseline results in Section 5.1 come from the first true end-to-end Colab run; the fine-tuned, upper-bound-LLM, and RAG tiers of the comparison table should be treated as pending until the corresponding notebook cells are confirmed complete. The README consolidation on the mentor repository's Group-39 branch was handed off as a ready-to-run cell but has not yet been confirmed executed.

9.1 Checkpoint 4 Gap — Closed at the Code Level

An audit of this project against the official brief found that Checkpoint 4's explicit requirement to "put their model into the RAG system and evaluate" had no working path in the codebase: `run_four_tier_comparison()` in `src/codegen_rag/rag/topk_experiment.py` reused the base model's `generate` function inside its `fine-tuned-model` branch, so a "fine-tuned + RAG" tier would have silently just re-run the base model; and the only RAG comparison path scored CodeBLEU alone into a table separate from the project's actual results/`comparison_table.csv`, so even a correct run would not have reached the deliverable. Both issues are now fixed: `run_four_tier_comparison()` takes an explicit `fine_tuned_generate_fn` parameter (with a logged warning if omitted, so the old failure mode can never happen silently again), and a new `RAGAugmentedTask` adapter (`src/codegen_rag/rag/rag_task_adapter.py`) lets any task module be retrieval-augmented and scored through the same `Evaluator.evaluate_task()` path as every other tier, landing full `exact_match/CodeBLEU/BERTScore` metrics in `comparison_table.csv`. Six new regression tests cover both fixes, bringing the suite to 153 passing tests. What remains is execution, not design: this fix needs to be run once in Colab with a GPU and the trained Rust checkpoint loaded so the `fine_tuned_rag` tier's real numbers populate the comparison table.

9.2 Spider vs. Spider 2.0 — Resolved

See Section 2.1: the brief's reference link points to Spider 2.0, but the project's actual code (verified directly in `src/codegen_rag/data/downloaders.py`) downloads and evaluates against the classic 2018 Spider benchmark, matching the project's own README citation. This is documented as a near-certain reference-list error in the brief rather than an implementation gap, with a one-line mentor confirmation recommended rather than a rebuild.

9.3 Remaining Open Item

The "AST retrieval (Jiang et al., 2023)" citation in the project README remains unverified against a specific paper and should be confirmed against the original project proposal before external submission.

10. Conclusion and Future Work

This project delivers a complete, tested, end-to-end pipeline for evaluating and enhancing a small open-source code language model, and — beyond the original research scope — a fully synchronized, tested, and published repository ready for mentor and grader review. This delivery phase closed three concrete gaps (correct checkpoint-aware model serving, genuine AST/dataflow-aware CodeBLEU scoring, and root-caused fixes for two non-obvious Colab/Google-Drive infrastructure issues) and produced a complete publishing and documentation trail across two GitHub repositories.

Natural next steps include running the newly fixed `fine_tuned_generate_fn` / `RAGAugmentedTask` path (Section 9.1) in Colab with a GPU and the trained Rust checkpoint loaded, so the `fine_tuned_rag` tier's real numbers populate `comparison_table.csv` alongside the small-LM baseline; completing the same full end-to-end run to populate the upper-bound-LLM and LLM+RAG rows; confirming the README consolidation has been run on the mentor repository; reproducing and resolving the PEFT "missing adapter keys" warning; confirming the Spider-vs-Spider-2.0 scope with mentors (Section 9.2); resolving the unverified AST-retrieval citation; and extending the fine-tuning approach demonstrated here for Rust to additional languages.

11. References

Nijkamp, E., et al. (2022). CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis.

Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. (Introduces both the Codex model and the pass@k metric.)

Feng, Z., et al. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages.

Pai, K. S., et al. (2024). CoDocBench: A Dataset for Code-Documentation Alignment in Software Maintenance. (Corrected from "Pal et al." in the project README.)

Yu, T., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.

Li, J., et al. (2023). Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD). NeurIPS 2023. (Corrected from "2024" in the project README.)

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

Lu, S., et al. (2021). ReACC: A Retrieval-Augmented Code Completion Framework.

Jiang, et al. (2023). AST-based code retrieval. Citation unverified against a specific publication — see Section 9.

Ren, S., et al. (2020). CodeBLEU: a Method for Automatic Evaluation of Code Synthesis.

Zhou, S., et al. (2023). CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code.

(pass@k is introduced within Chen et al., 2021, above, rather than a separate publication.)